

Dino Game Bot

A Brief: What It Does, How It Decides, and Where It Is Soft

Portfolio explainer: concise overview

The project in four sentences

Chrome hides an endless-runner game at `chrome://dino/`: a dinosaur runs, cacti approach, you press space to jump. This project plays it without touching the game’s code or data, by screenshotting one rectangle of the screen and asking whether any pixel in it is the colour an obstacle would be. If one is, it sends a space keypress. 812 lines of Python, three files, three dependencies.

Contents

1	The shape of the thing	2
2	Why step 1 exists	2
3	The three good decisions	2
3.1	Demo mode patches the eye, not the brain	2
3.2	The fixture imports the constants it tests against	3
3.3	A change was only kept because it could be proven	3
4	Where it is soft	3
5	Verified against not verified	5
6	Glossary	5

1 The shape of the thing

Everything the bot does hangs off one loop, and the loop is four steps long.

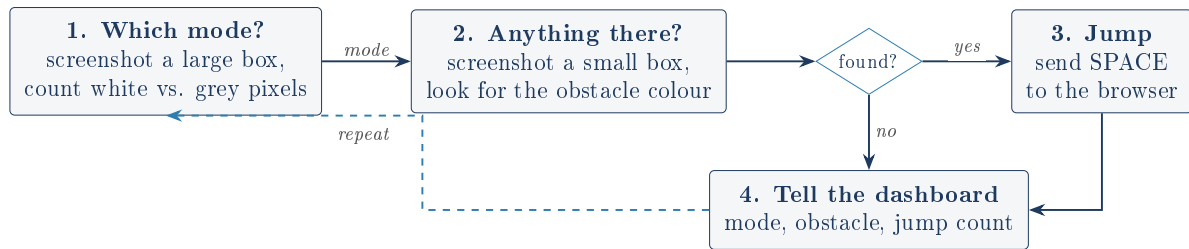


Figure 1: The whole bot. Step 1 exists only because step 2’s answer depends on it.

2 Why step 1 exists

The Dino game inverts its palette periodically: day mode is dark sprites on a white background, night mode is white sprites on a dark background. So “what colour is an obstacle?” has two answers, and the wrong one is not merely inaccurate, it is inverted: a bot hunting for dark pixels in night mode would find the background everywhere and jump forever.

Hence the sequence. `check_day` takes the majority vote of white against dark grey across a large region, which settles the mode; `check_obstacle` then hunts for whichever colour the *other* mode uses, inside a much smaller region placed just in front of the dinosaur.

The intelligence is in the rectangle, not the code

`check_obstacle` does not measure distance, speed, or shape. It asks whether a single pixel of the target colour appears anywhere in its box. “Obstacle in the box” and “jump now” are treated as the same event, which is only true because the box was hand-placed at exactly the right spot. That makes the four hardcoded coordinates the most load-bearing and most fragile thing in the repository.

3 The three good decisions

Worth knowing, because each is defensible in an interview.

3.1 Demo mode patches the eye, not the brain

The bot needs a live Chrome playing a live game on a screen matching its hardcoded coordinates. A fresh clone has none of that, so `--demo` replays six pre-drawn frames instead. The interesting part is *how*.

```

1 original_grab = ImageGrab.grab
2 try:
3     for filename, frame in frames:
4         ImageGrab.grab = lambda bbox=None, _frame=frame: _frame.crop(bbox)
5         is_day = check_day()
6         obstacle = check_obstacle(is_day)
7         ...
8 finally:
9     ImageGrab.grab = original_grab
  
```

Listing 1: The substitution, from `run_demo`

Rather than adding an `if demo:` branch inside the detection functions, it swaps out the screenshot call those functions make, `ImageGrab.grab`, for a stand-in that crops a stored image. The

detection functions are byte-for-byte the ones live mode runs, and contain no code that could tell the difference.

Why that distinction is the whole point

A demo-only branch means demo mode tests a path production never takes, which is how tests come to pass while the product is broken. Patching the input leaves the logic untouched, so what runs really is what ships. The **finally** restore matters too: without it the process would keep returning a stale dino frame from any later capture.

3.2 The fixture imports the constants it tests against

`tools/generate_demo_frames.py` could have hardcoded the colours and coordinates when drawing its frames. It imports them from `main.py` instead, so re-measuring the coordinates for a different screen moves the test frames with them. A fixture that duplicates the constants of the code it tests is a fixture that will eventually disagree with it silently.

3.3 A change was only kept because it could be proven

`check_day` and `check_obstacle` gained an optional `tolerance` argument: instead of requiring a pixel to be bit-for-bit (255,255,255), count it if every channel is within `tolerance`. This matters because real screen captures drift (anti-aliasing, dithering, compression) in ways a hand-drawn flat colour never does.

`tools/compare_detection_robustness.py` tests it properly: both logics against both a clean and a deliberately noisy frame set, so “did it help?” and “did it break what worked?” are answered by the same run, with the expected answers derived from filenames rather than a hand-maintained table.

	clean frames	noisy frames
old (<code>tolerance=0</code>)	6/6 correct	4/6 correct
new (<code>tolerance=8</code>)	6/6 correct	6/6 correct

Table 1: I ran this. The README’s table reproduces exactly, including which two frames the old logic fails.

And it defaults to 0, the original exact-match behaviour, because it has only ever been proven against synthetic noise. Offering a change rather than imposing it, on the grounds that the evidence is synthetic, is the right instinct.

4 Where it is soft

The findings below come from running the code, not reading it.

The noise in the benchmark only applies to one of the two colours

`dither()` shifts every channel by ± 6 and its docstring promises “no pixel in the output is ever exactly equal to a color present in the input”. It is not true. Dark grey 83 moves to 89 or 77, both genuinely off-target. White 255 moves to 249 or to `min(255, 261)`, which is 255 again. Measured: 420,000 pixels of the noisy `01_day_clear.png`, exactly half, come through bit-for-bit white. So the noise fully perturbs grey and half-ignores white, and the experiment is not measuring what it thinks. Fixing it is one line, and would likely make the tolerance feature look better rather than worse, but as written the evidence does not support the conclusion it is offered for.

A tie in `check_day` silently votes “night”

The return is `day_count > night_count`. A frame containing neither colour scores `0 > 0`, which is `False`, so the function reports night on zero evidence. Verified against a uniform grey frame. This is why two of the old logic’s four “correct” answers on the noisy set are worthless: 04 and 05 are night frames, the old logic saw nothing at all, defaulted to night, and was scored correct. Its honest score on that set is closer to 2/6 than 4/6. The benchmark cannot distinguish “right” from “no opinion, guessed, got lucky”.

The recommended default would slow the bot roughly ninefold

The README’s “What I’d do differently” proposes turning tolerance on by default. Nothing anywhere measures what that costs. I did, on this machine, excluding the real screen grab: one loop iteration’s pixel work goes from about **70 ms** at `tolerance=0` to about **608 ms** at `tolerance=8`, roughly 14 Hz down to 1.6 Hz. The cause is that `count()` and `in` are C-speed list scans, while the tolerant path is a Python generator calling `_color_distance` once per pixel, about 140,000 function calls where there was one C loop. A bot forming an opinion 1.6 times a second will hit obstacles while still counting pixels from a stale frame. The feature is correct; accuracy was simply the only axis measured, and for a real-time loop it is not the deciding one. It is fixable (do the comparison in C via Pillow’s `point()`, or sample a sparse grid instead of 140,400 pixels), but “turn it on by default” as stated would break the bot.

The advertised 10 ms polling loop actually runs near 14 Hz

The README frames the 10 ms sleep as the reaction-time knob. It is not: it sits next to roughly 70 ms of unavoidable pixel scanning, and it only executes on iterations that jumped. On a clear track the loop never sleeps at all, pinning one core at 100% while it takes two screenshots and scans 206,000 pixels per pass. The real lever is the pixel count. `check_day` scans 140,400 pixels on every iteration to answer one binary question that a few hundred would answer, about a palette that flips maybe once a minute. Caching the mode would remove roughly 70% of the loop’s cost for no loss.

The demo frames omit the feature most likely to break the bot

`make_frame` fills the canvas with one flat colour, and says so: “the real game has sky and a thin ground strip”. That simplification is harmless for `check_day`, which counts and compares. It is not harmless for `check_obstacle`, which fires if the target colour appears *once* in its box. In the real day-mode game the ground line is dark grey, the exact colour `check_obstacle` hunts for in day mode, and the box spans the vertical band where a ground line would plausibly sit. If it falls inside, the bot jumps continuously forever. To be precise: that the frames contain no ground strip is **confirmed** (their histograms hold two colours, no third); that the real game’s ground line intersects the box is **inferred** and untestable here, with no Chrome and no live game. The point survives either way: the fixture cannot tell those two worlds apart, so “six of six pass” proves less than it sounds. The fixture was drawn by the same person holding the same assumption as the code.

Smaller items

The constants named `DAY_GROUND_COLOR/NIGHT_GROUND_COLOR` hold background colours, not ground colours, and the docstrings inherit the confusion. On a headless Linux box that is not running as root, there is no working way to stop the loop: the console view has no `closed` attribute, and the `q` hotkey needs root to read the input device. `driver.find_element(value="t")` is re-queried on every jump instead of hoisted. And two

of the README’s eight “Challenges” bullets are about LaTeX errors in the explainer PDFs, which are not challenges of this project at all.

5 Verified against not verified

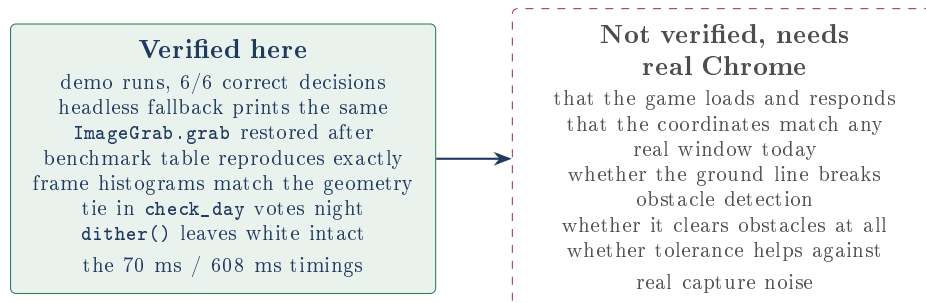


Figure 2: The line this project cannot cross without a machine running the game. The README draws the same line honestly; this document adds the measurements.

6 Glossary

RGB

A colour as three numbers 0 to 255: red, green, blue. White is (255,255,255); the game’s dark grey is (83,83,83).

Bounding box

A screen rectangle as (left, top, right, bottom) in pixels from the top-left corner. Down is positive.

Screen scraping

Driving a program by reading its pixels and sending fake input, because it offers no API. Fragile by nature: the only contract is that the picture keeps looking the same.

Selenium / WebDriver

A library that drives a real browser, through a helper program (`chromedriver`). Since 4.6 it fetches that helper itself, which is why no manual driver setup is needed.

Pillow / ImageGrab

Python’s imaging library; `ImageGrab.grab(bbox)` captures a screen rectangle. It is the bot’s eye and the one thing demo mode replaces.

Monkeypatching

Replacing a module’s function at runtime from outside it. Demo mode’s mechanism.

tkinter / mainloop()

Python’s built-in windowing toolkit. `mainloop()` never returns, so instead the dashboard pumps the event queue once per iteration and returns immediately, keeping the window alive without taking the loop’s thread.

deque(maxlen=n)

A queue that discards its oldest item automatically when full. Makes “keep the last 8 events” a data-structure choice.

Chebyshev distance

Difference between two colours measured as the largest gap on any single channel. What **tolerance** compares against.

The summary

A small project that does one thing, unusually well-instrumented for its size, with three genuinely good instincts: exercise production code from the test path, keep one source of truth for the constants, and refuse a change the evidence does not support. Its soft spot is that the evidence it does trust measures accuracy on frames whose noise is half-broken, never measures speed at all, and cannot see the failure mode most likely to be real. All of that is cheap to fix, and much better learned here than from an interviewer.